RK2206实训箱操作手册—ENS160开发

VEML7700光照传感器

1. 目录

- 实验内容
- ENS160 API说明
  - 函数说明
    - ENS160 初始化
    - ENS160 设置温湿度补偿
    - ENS160 读取 eCO2 数据
  - 常量和枚举类型说明
    - I2C 端口
    - 运行模式（ens160_modes_t）
  - 寄存器地址定义
- 代码编译
  - 添加文件
  - 修改 BUILD.gn 文件
  - 编译结果
  - 烧录固件
  - 运行结果
- 注意事项
- 技术规格

2. 实验内容

在RK2206实训箱上使用I2C接口，实现ENS160二氧化碳传感器的编程开发。

3. ENS160 API说明

I2C相关API详见PPT。

3.1 函数说明

3.1.1 (一)、ENS160 初始化

定义:	**ens160_init(void)**
功能:	**初始化 ENS160 传感器并进入工作模式**
参数:	**void**
返回值:	**void**
依赖:	**\code_base\b4_i2c_ens160\include\ens160.h**

定义:	**ens160_init(void)**
Tips:	**初始化 I2C0_M2（400KHz），并设置设备运行模式为 Operational**

Table 1

3.1.2 (二)、ENS160 设置温湿度补偿

定义:	**ens160_set_temperature_humidity(double temperature, double humidity)**
功能:	**为 ENS160 设置环境温度与湿度，用于补偿**
参数:	`temperature`**: 温度(°C);** `humidity`**: 相对湿度(%RH)**
返回值:	**void**
依赖:	**\code_base\b4_i2c_ens160\include\ens160.h**
Tips:	**内部按数据手册转换为寄存器格式（温度 K×64、湿度 %×512)**

Table 2

3.1.3 (三)、ENS160 读取 eCO2 数据

定义:	**ens160_get_data(uint16_t *data)**
功能:	**读取 ENS160 的 eCO2 数据**
参数:	`data`**: 指向存放 eCO2(ppm) 的指针**
返回值:	**void**
依赖:	**\code_base\b4_i2c_ens160\include\ens160.h**
Tips:	**函数内部先写寄存器地址后再读 2 字节数据并拼接为 16 位值**

Table 3

3.2 常量和枚举类型说明

3.2.1 I2C 端口

- `EI2C0_M2`

3.2.2 运行模式（ens160_modes_t)

- `DEEP_SLEEP_MODE` (0x00)
- `IDLE_MODE`
- `OPERATIONAL_MODE`

3.3 寄存器地址定义

名称	地址	功能描述
`PART_ID`	0x00	器件 ID/芯片识别号

名称	地址	功能描述
`OPMODE`	0x10	运行模式选择（Deep Sleep/Idle/Operational）
`CONFIG`	0x11	配置寄存器（功能与行为配置）
`COMMAND`	0x12	命令寄存器（执行特定操作/复位/触发等）
`TEMP_IN`	0x13	温度补偿输入（单位：K×64 的无符号整数）
`RH_IN`	0x15	湿度补偿输入（单位：%×512 的无符号整数）
`DEVICE_STATUS`	0x20	设备状态（错误/就绪/新数据标志等）
`DATA_AQI`	0x21	空气质量指数 AQI（分级/指数值）
`DATA_TVOC`	0x22	总挥发性有机物 TVOC 数据
`DATA_ECO2`	0x24	等效二氧化碳浓度 eCO2 数据（ppm）
`DATA_T`	0x30	传感器内部温度读数（格式依器件定义）
`DATA_RH`	0x32	传感器内部相对湿度读数（格式依器件定义）
`DATAMISR`	0x38	数据一致性/校验相关寄存器（用于数据有效性）
`GPR_WRITE`	0x40	通用目的寄存器写入口（固件/算法交互）
`GPR_READ`	0x48	通用目的寄存器读出口（固件/算法交互）

Table 4

4. 代码编译

4.1 添加文件

将`~vendor/company_demo/b4_i2c_ens160`这整个文件夹拖入`~/application/samples`路径下

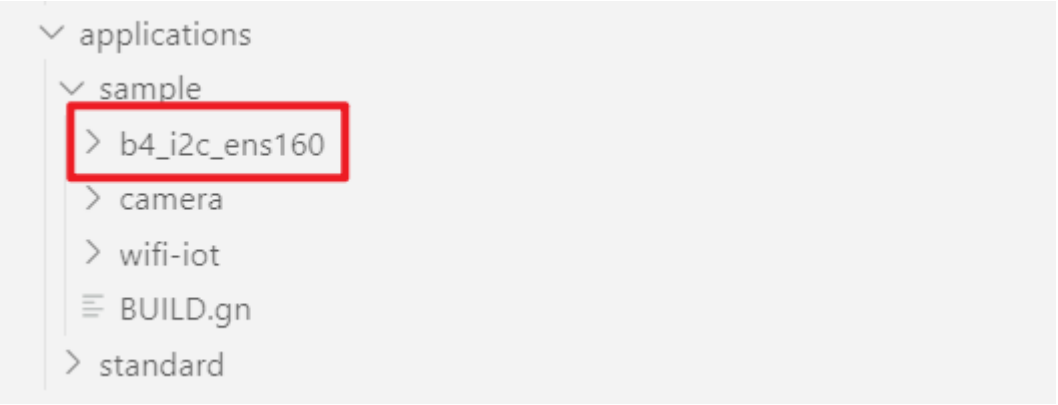


Figure 1

4.2 修改 BUILD.gn 文件

修改 `~/application/samples/` 路径下 BUILD.gn 文件，将如下代码添加到对应位置，指定 `ens160_example` 参与编译。

```
"./b4_i2c_ens160:ens160_example",
```

Fence 1

```
import("//build/lite/config/component/lite_component.gni")

lite_component("app") {
    features = [
        "./b4_i2c_ens160:ens160_example",
    ]

    deps = [
    ]
}
```

Figure 2

修改 `device/lockzhiner/rk2206/sdk_liteos` 路径下 Makefile 文件，添加 `-lens160_example` 参与编译。

```
hardware_LIBS = -lhal_iothardware -lhardware -lshellcmd -lens160_example
```

Fence 2

```
#######################################
# LDFLAGS
#######################################
LDSCRIPT = board.ld

boot_LIBS = -lbootstrap -lbroadcast
hardware_LIBS = -lhal_iothardware -lhardware -lshellcmd -lens160_example
hdf_LIBS = -lhdf_config -lhdf_core -lhdf_osal_lite -lhdf_platform_lite \
#-lsample_driver -lgpio_driver -lpwm_driver -li2c_driver -ladc_driver -lspi_driver
```

Figure 3

4.3 编译结果

```
root@05fdda9d5ead: /home/
[OHOS INFO] [829/851] STAMP obj/kernel/liteos_m/components/fs/fs.stamp
[OHOS INFO] [830/851] AR libs/libshell.a
[OHOS INFO] [831/851] AR libs/libhichainsdk.a
[OHOS INFO] [832/851] STAMP obj/base/security/deviceauth/frameworks/deviceauth_lite/hichainsdk.stamp
[OHOS INFO] [833/851] gcc cross compiler obj/third_party/paho_mqtt/MQTTClient-C/src/libpahomqtt_static.MQTTClient.o
[OHOS INFO] [834/851] AR libs/libpahomqtt_static.a
[OHOS INFO] [835/851] AR libs/liblwip.a
[OHOS INFO] [836/851] STAMP obj/kernel/liteos_m/kernel.stamp
[OHOS INFO] [837/851] ACTION //test/xts/acts/build_lite:acts_xdevice(//build/lite/toolchain:arm-none-eabi-gcc)
[OHOS INFO] [838/851] STAMP obj/test/xts/acts/build_lite/acts_xdevice.stamp
[OHOS INFO] [839/851] ACTION //test/xts/acts/build_lite:acts_generate_module_data(//build/lite/toolchain:arm-none-eabi-g
cc)
[OHOS INFO] [840/851] ACTION //vendor/lockzhiner/rk2206/hdf_config:hc_gen(//build/lite/toolchain:arm-none-eabi-gcc)
[OHOS INFO] [841/851] STAMP obj/test/xts/acts/build_lite/acts_generate_module_data.stamp
[OHOS INFO] [842/851] STAMP obj/vendor/lockzhiner/rk2206/hdf_config/hc_gen.stamp
[OHOS INFO] [843/851] gcc cross compiler obj/out/rk2206/lockzhiner-rk2206/gen/vendor/lockzhiner/rk2206/hdf_config/libhdf
_config.hdf_hex.o
[OHOS INFO] [844/851] AR libs/libhdf_config.a
[OHOS INFO] [845/851] STAMP obj/device/rockchip/rk2206/sdk_liteos/sdk.stamp
[OHOS INFO] [846/851] STAMP obj/device/rockchip/rk2206/sdk_liteos/wifiiot_sdk.stamp
[OHOS INFO] [847/851] ACTION //test/xts/acts/build_lite:acts(//build/lite/toolchain:arm-none-eabi-gcc)
[OHOS INFO] [848/851] STAMP obj/test/xts/acts/build_lite/acts.stamp
[OHOS INFO] [849/851] STAMP obj/build/lite/ohos.stamp
[OHOS INFO] [850/851] ACTION //device/rockchip/rk2206/sdk_liteos:liteos(//build/lite/toolchain:arm-none-eabi-gcc)
[OHOS INFO] [851/851] STAMP obj/device/rockchip/rk2206/sdk_liteos/liteos.stamp
[OHOS INFO] /home/openharmony/vendor/lockzhiner/rk2206/fs.yml not found, stop packing fs. If the product does not need t
o be packaged, ignore it.
[OHOS INFO] lockzhiner-rk2206 build success
[OHOS INFO] cost time: 0:01:40
root@05fdda9d5ead:/home/openharmony#
```

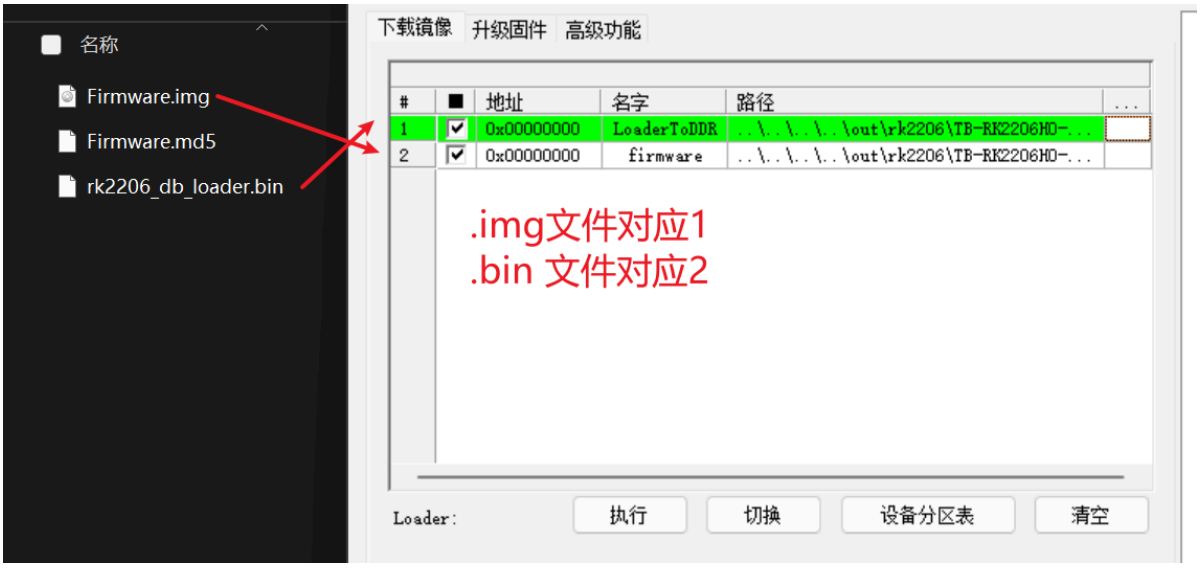
Figure 4

4.4 烧录固件

打开烧录工具RKDevTool

Figure 5

指定两个烧录文件, 编译出的目标文件在 `out/rk2206/lockzhiner-rk2206/images` 目录下。



在RK2206实训箱上上，长按左侧模块控制终端的MaskRom按键（MaskRom按键不要松开），点击ReSet按键（短按，按下后松开，但MaskRom按键不要松开，大约5秒后松开)，烧写工具出现：“发现一个MASKROM设备”。如下图所示：

Figure 6

此时点击“执行”，下载程序，下载完成如下：

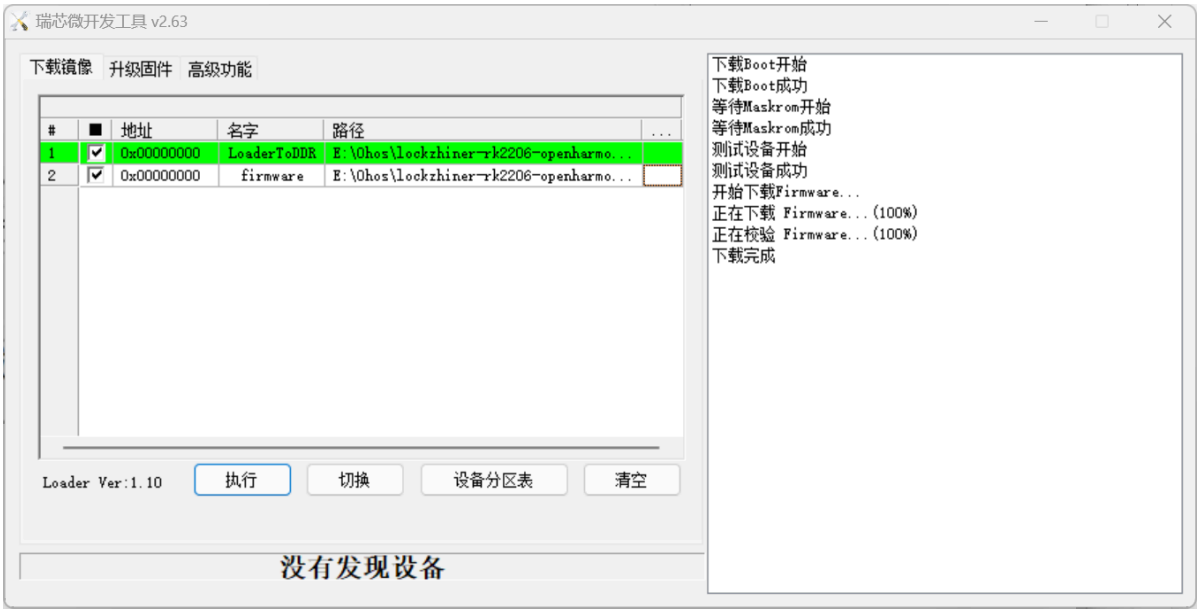


Figure 7

4.5 运行结果

示例代码编译烧录代码后，按下开发板的RESET按键，连接串口，可以看到二氧化碳浓度信息。

```
entering kernel init...
hilog will init.
[MAIN:D]Main: LOS_Start ...
Entering scheduler
OHOS # hiview init success.
[GPIO:D]LzGpioInit: id 1 is initialized successfully
[GPIO:D]LzGpioInit: id 0 is initialized successfully
co2: 400
co2: 400
co2: 400
co2: 400
co2: 400
co2: 400
```

Fence 3

```
entering kernel init...
hilog will init.
[MAIN:D]Main: LOS_Start ...
Entering scheduler
OHOS # hiview init success.[GPIO:D]LzGpioInit: id 1 is initialized successfully
[GPIO:D]LzGpioInit: id 0 is initialized successfully
co2: 517
co2: 522
co2: 498
co2: 524
co2: 518
co2: 563
co2: 1750
co2: 1174
co2: 1013
co2: 920
co2: 870
```

人为呼气干扰

Figure 8

5. 注意事项

1. **初始化顺序**: 必须先调用 `ens160_init()` 再进行补偿/读取
2. **I2C 通信**: 设备地址 `0x52`，总线频率 400KHz
3. **补偿建议**: 提供实时温湿度能改善测量准确度
4. **数据单位**: `ens160_get_data` 返回 eCO2，单位 ppm

6. 技术规格

- **通信接口**: I2C
- **I2C 地址**: 0x52
- **输出**: eCO2、TVOC（本驱动示例读取 eCO2）
- **运行模式**: Deep Sleep / Idle / Operational